

IEOR 4578 — FORECASTING PROJECT

Retail Forecasting

Luca Barattini · Federico Giorgi · Zoe Lucas

May 5, 2026

Contents

1	Business Context & Objective	3
1.1	Introduction	3
1.2	Retail Operations & Inventory Dynamics	3
1.3	The Cost of Forecast Error: Stockouts and Overstock	3
1.4	High-Level System Architecture	4
1.5	Repository Structure & MLOps Implementation	5
1.6	The Agentic AI Layer: Orchestrating the Complexity	6
2	Project Evolution & Methodology Upgrades	7
2.1	Evaluation of the Baseline Framework	7
2.2	Resolving Conceptual Errors and Code Logic Gaps	7
2.3	Architectural Overhaul and Roadmap	8
3	Data Strategy & Engineering	9
3.1	Data Extraction	9
3.2	Target Variable	9
3.3	Predictive Variables	9
3.4	Data Processing & Feature Engineering Pipeline	11
4	Modeling	17
4.1	Strategy	17
4.2	Ridge Regression	18
4.2.1	Scaling and Feature Engineering	18
4.2.2	Inference and Evaluation	18
4.3	Prophet	20
4.3.1	Model Configuration and Feature Engineering	20
4.3.2	Inference and Evaluation	20
4.4	LightGBM	22
4.4.1	Native Categorical Handling and The Tweedie Objective	22
4.4.2	Training and Inference	22
4.5	SARIMAX (Seasonal ARIMA with eXogenous Variables)	24
4.5.1	Model Configuration and Hyperparameter Tuning	24
4.5.2	Inference, Evaluation, and Serialization	24
4.6	Non-Stationary Transformer	26
4.6.1	PyTorch Architecture and De-Stationary Attention	26
4.6.2	Data Engineering and Tensor Construction	27
4.6.3	Inference, Inverse Scaling, and MLOps Artifacts	27
5	The Agentic Layer: AI Forecasting Assistant	29
5.1	Agent Architecture and The ReAct Framework	29
5.2	Tool Integration and Context Retrieval	29
5.3	Prompt Engineering and Business Interpretation	30
5.4	Example Interaction	30

6	Results & Evaluation	32
6.1	Evaluation Metrics: A Retail Supply Chain Perspective	32
6.2	Model Performance & Benchmarking	33
6.3	Temporal Error Distribution	34
7	Improvements and Next Steps	36
7.1	Evolution from the Baseline Framework	36
7.2	Key Technical Enhancements	36
7.3	Next Steps and Strategic Roadmap	37
8	Team	38
A	Appendix: Individual SKU Error Spread Distributions	39
A.1	LightGBM (State-of-the-Art Tree Baseline)	39
A.2	Non-Stationary Transformer	39
A.3	Ridge Regression	40
A.4	SARIMAX	40
A.5	Facebook Prophet	41

1 Business Context & Objective

1.1 Introduction

This report documents the end-to-end machine learning pipeline developed to forecast weekly SKU-level demand for a UK-based non-store online retailer. Utilizing the Online Retail II dataset, comprising transactional records from December 2009 to December 2011, we process raw event-driven sales data through rigorous feature engineering, multi-dimensional clustering, and advanced predictive modeling.

The primary objective is to produce and benchmark reliable demand projections across the active product portfolio, supporting tactical inventory replenishment and revenue planning over a 12-week-ahead horizon. Instead of treating all products equally, our framework segments the catalog by seasonal behavior, sales volume, and semantic identity to deploy tailored forecasting models. All models are rigorously evaluated on a chronologically held-out test set, utilizing the Weighted Mean Absolute Percentage Error (WMAPE) to ensure a robust, volume-weighted comparison that aligns with true business impact.

Furthermore, to bridge the gap between data science and supply chain operations, we implement an **Agentic AI Assistant**. This interface allows an inventory manager to query forecasts by product ID or semantic category via natural language, automatically relying on an Auto-Routing algorithm to select the mathematically optimal predictive model for that specific item.

1.2 Retail Operations & Inventory Dynamics

Unlike continuous utilities, retail demand, particularly in a wholesale and gift-ware context, is highly discrete, volatile, and subject to extreme Pareto skew (the 80/20 rule). The company operates by selling unique, all-occasion items to both individual consumers and wholesalers. This creates a complex demand landscape characterized by frequent *intermittent sales*: long periods of zero demand interspersed with sudden, high-volume bulk orders.

To support replenishment planning and safety stock calibration, demand must be aggregated from raw, timestamped transactions into standardized weekly cycles. Consequently, supply chain managers require highly granular forecasts, calculating distinct, multi-week projections for thousands of operationally relevant SKUs to construct a complete tactical schedule.

The forecasting challenge in this environment is exacerbated by product churn and seasonality. The continuous introduction of new items creates a "Cold-Start" problem, while the dominance of holiday-driven purchasing behavior necessitates models capable of capturing autoregressive momentum, shifting pricing strategies (such as promotional discounts), and long-term yearly cyclicity.

1.3 The Cost of Forecast Error: Stockouts and Overstock

Beyond simple statistical accuracy, forecasting serves a critical financial purpose. In retail, demand projections dictate purchasing decisions, capital allocation, and warehouse space for the upcoming quarter. When an inventory manager's forecast deviates from the actual consumer demand,

the business incurs severe financial penalties that directly erode profit margins. These penalties manifest in two distinct scenarios:

- **Over-forecasting (Overstock):** The retailer procures more units than the market demands. This results in capital tied up in stagnant inventory, incurring continuous warehousing and holding costs. Eventually, to clear the surplus, the retailer is forced to apply aggressive markdowns, selling the inventory at a substantial financial **loss**.
- **Under-forecasting (Stockout):** The retailer procures too few units. When inventory is depleted, the business cannot fulfill incoming orders. This directly translates to **lost revenue** and, more critically in a B2B wholesale environment, damages relationships, leading to long-term customer churn.

Value Proposition: By deploying robust, data-driven machine learning models, inventory managers can optimize safety stock, minimize exposure to both holding costs and stockout risks, and protect the company's bottom line.

1.4 High-Level System Architecture

To synthesize the project's scope, Figure 1 presents the end-to-end system architecture.

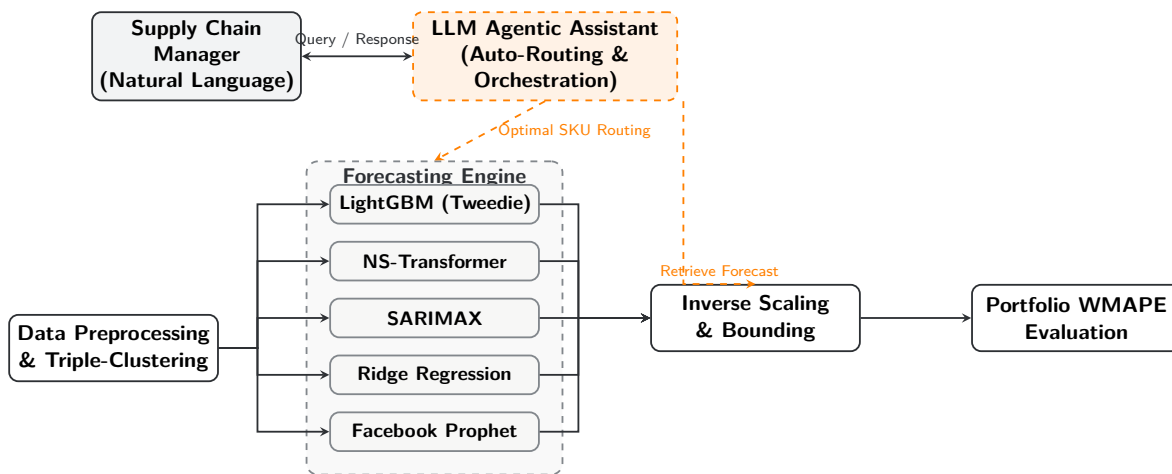


Figure 1: The high-level system architecture. Centralized data preprocessing feeds into five parallel forecasting models. The LLM Agent acts as an orchestrator, connecting the business user to the predictive backend via an Auto-Router.

The pipeline begins with centralized data preprocessing and behavioral clustering, ensuring a standardized, clean input for all downstream tasks. The core forecasting engine comprises five distinct modeling architectures running in parallel: LightGBM, the Non-Stationary Transformer, SARIMAX, Ridge Regression, and Facebook Prophet. Each model generates cluster-level predictions in a mathematically scaled space. These are subsequently inverse-transformed (Safe Inverse) and merged back to the individual SKU level to evaluate the overall Portfolio WMAPE.

Sitting above this predictive layer is the Agentic AI Assistant, which acts as the orchestration interface. It interprets natural language queries from supply chain managers, dynamically routes

requests to the mathematically optimal model via a `.json` lookup table, and returns actionable forecasts, effectively bridging the gap between complex MLOps and business decision-making.

1.5 Repository Structure & MLOps Implementation

The conceptual architecture presented in Figure 1 is physically implemented through a modular Python codebase. To ensure scalability and maintainability, the repository follows a strict separation of concerns, organized as follows:

Directory	Pipeline Step / Purpose
<code>src/tools/</code>	Core utility modules for data ingestion (<code>data_loader.py</code>), transaction cleaning, NLP embeddings, clustering, and evaluation metrics.
<code>src/models/</code>	Standardized modeling modules implementing a unified API for Optuna tuning, training, prediction, and SKU-level inverse scaling.
<code>scripts/</code>	Executable automation scripts (e.g., <code>process_data.py</code>) to run the end-to-end feature engineering pipeline in batch mode.
<code>notebooks/</code>	Interactive playgrounds used for visualization, benchmarking, and exploratory analysis.
<code>agent/</code>	The production inference layer containing the LLM chatbot interface and dynamic auto-routing prediction logic.

Table 1: Physical organization of the project components.

This modular approach ensures that each component, from the Data Preprocessing block to the Forecasting Engine, can be updated or replaced independently, reflecting modern MLOps best practices.

Pipeline Artifacts The preprocessing and modeling phases produce consolidated datasets and serialized objects. All categorical definitions are natively embedded into the final Parquet dataset, while trained models are serialized for rapid inference:

Artifact	Description
<code>processed_retail_data.parquet</code>	The core ML-ready dataset integrating all temporal variables, dynamic pricing flags, NLP cluster mappings, and autoregressive features.
<code>agent/artifacts/*.pkl</code>	Serialized model state dictionaries, and optimal SKU-to-Model routing matrices. These enable instant predictions by the AI Agent.

Table 2: Summary of generated data and model artifacts.

1.6 The Agentic AI Layer: Orchestrating the Complexity

Sitting above the predictive engine described in the previous section is the **Agentic AI Assistant**. This layer is designed specifically for non-technical supply chain managers, fulfilling a crucial UX requirement: abstracting the complexity of multiple models and behavioral clusters into a single conversational interface.

Instead of navigating complex data pipelines, the manager interacts with the system via a natural language prompt or a direct StockCode query. The LLM-based Agent acts as an intelligent intermediary; it retrieves the product's historical and semantic profile, utilizes the 'auto' router to trigger the lowest-error model, and returns real-time visualizations and interpretations. This agentic approach transforms raw predictive power into a decision-support tool, allowing for real-time visualization and interpretation of sales trends without requiring expertise in the underlying machine learning frameworks. The following sections detail the data engineering, clustering strategies, and modeling architectures that enable this multi-model agentic ecosystem.

2 Project Evolution & Methodology Upgrades

2.1 Evaluation of the Baseline Framework

This project represents a comprehensive continuation, refinement, and architectural overhaul of the baseline forecasting notebook initially delivered by Team “Trendsetters”. While their foundational work demonstrated a solid academic structure and a highly pragmatic approach to business implications, such as evaluating errors over 4-week aggregated blocks to smooth out demand volatility, an in-depth code review revealed several critical methodological gaps and unfulfilled core requirements.

Specifically, the delivery missed three fundamental pillars of the original assignment specification:

- **Absence of Clustering.** The project brief explicitly required that teams “*clustered consumers on their usage patterns, or products in similar groups of products, and then built models for each cluster.*” The baseline code completely ignored this clustering mandate. Instead, it executed a monolithic for loop over the top 250 revenue-generating SKUs, training 250 completely independent and isolated models. While the team calculated the Coefficient of Variation (CV) to categorize SKUs into “LowVol”, “MidVol”, and “HighVol”, this was only used retroactively for an optional appendix diagnostic, never for actual algorithmic clustering. Consequently, the models missed the opportunity to learn shared cross-SKU seasonal patterns, significantly increasing the computational burden.
- **Omission of Exogenous Variables.** To reduce errors, the assignment suggested to “*create time-based features, incorporate holidays or other schedules, and add external data, such as weather...*” The baseline explicitly stated that no external regressors were included. The data preprocessing only aggregated historical Quantity and Revenue, feeding a purely univariate time-series into the models. By omitting variables like UK holidays, pricing elasticity, or promotional flags, the models remained entirely “blind” to external retail drivers, which directly contributed to excessive forecasting errors during peak holiday shopping periods.
- **A “Fake” Agentic Layer.** The business requirement mandated an Agentic AI UX where a user could “*enter the product name/id... in a prompt and ask the LLM to provide you with the latest forecasts. The LLM will sort of run a mapping...*” The baseline report claimed to feature an “Agentic Forecast Retrieval Layer.” However, the codebase (`extract_sku_from_prompt`) revealed a simple Python function relying on hardcoded Regular Expressions (`re.findall(r"[A-Za-z0-9]+", prompt.upper())`) to perform an exact string match against a predefined list. There was no actual Large Language Model (LLM) instantiated, no natural language reasoning, and no dynamic tool-calling involved. It was merely a standard search bar masquerading as an AI agent.

2.2 Resolving Conceptual Errors and Code Logic Gaps

Beyond the missing assignment requirements, our code audit uncovered severe conceptual flaws in the mathematical modeling that necessitated a complete rewrite of the forecasting engine. These logic gaps heavily distorted the baseline’s reported accuracy:

- **The “Fake” SARIMA & Forecast Collapse.** The baseline claimed that SARIMA was their strongest model. However, an inspection of their `model_sarima_safe` function reveals the following instantiation: `SARIMAX(train, order=(1,1,1), seasonal_order=(0,0,0,0))`. By forcing the seasonal period to zero, they mathematically disabled the 'S' in SARIMA, reducing it to a standard, non-seasonal ARIMA(1,1,1) model. In retail forecasting, seasonality is arguably one of the most critical component. Because their model had no seasonal memory, the multi-step autoregressive forecasts quickly collapsed into a flat, constant mean. Their final "12-week forecast" was effectively a 2-to-4 week forecast padded with identical flat values for the remainder of the horizon, rendering it useless for actual supply chain planning.
- **Silent Model Fallbacks.** The baseline codebase contained highly dangerous exception handling. Within both their SARIMA and LightGBM functions, as well as the `rolling_block_evaluate` loop, they utilized `broad try... except Exception: blocks`. If a model failed to converge, exploded geographically (`np.abs(fcst).max() > 1e6`), or produced NaN values, the code *silently* overwrote the prediction with a basic Naïve model (carrying forward the last known value). This approach completely masked true algorithmic failures. When the evaluation metric (MAPE) was calculated, it was often scoring the Naïve fallback rather than the complex algorithm itself. This corrupted the model selection phase and actively misled business stakeholders regarding the stability and reliability of the deployed machine learning pipeline.

2.3 Architectural Overhaul and Roadmap

To address these limitations, we completely restructured the project, transitioning from a single, monolithic, and error-prone Jupyter Notebook to a robust, deployment-ready repository. The new MLOps-aligned architecture enforces a strict separation of concerns, dividing the codebase into dedicated modules as described in section 1.5.

By moving away from the flawed baseline, this report details the implementation of a vastly superior forecasting ecosystem. Our refined methodology achieves the following:

1. **Exogenous Features:** We engineer robust exogenous signals (pricing elasticity, UK holidays) to break the accuracy ceiling of the baseline models.
2. **Shift to Cluster-Based Modeling:** By implementing a multi-dimensional clustering architecture, we train single global models per cluster rather than per SKU. This drastically reduces computational overhead while allowing the models to learn cross-SKU seasonal patterns.
3. **Integration of Modern Architectures:** We deploy new state-of-the-art algorithms, including Facebook Prophet and Non-Stationary Transformers.
4. **Deployment of a True Agentic Layer:** We replace the Regex placeholder with a genuine LangChain/LLM orchestration layer capable of interpreting natural language and dynamically mapping users to the correct cluster artifacts, as introduced in 1.6.
5. **Enhanced Metrics and Exogenous Features:** We introduce Volume-Weighted MAPE (WMAPE) to accurately reflect business impact.

3 Data Strategy & Engineering

3.1 Data Extraction

The raw dataset, **Online Retail II**, contains transactional records for a UK-based and registered non-store online retail company between December 1, 2009, and December 9, 2011. The company mainly sells unique all-occasion gift-ware to both individual customers and wholesalers. The initial extraction yields a massive, event-driven ledger containing over 1 million transactional rows.

Property	Value
Time range	2009-12-01 to 2011-12-09
Unique Products	4,873 active SKUs
Measurement frequency	Event-driven (Transactional timestamps)
Raw shape (long format)	1,067,371 rows $\times$ 8 columns
Format	Excel (.xlsx)

Table 3: Raw Dataset Summary

Data Loading & Quality Checks To ensure architectural consistency and reproducibility, data ingestion is encapsulated within the centralized `load_raw_data` utility. Retail exports data consist of discrete, event-driven logs. Therefore, our pipeline explicitly handles the heavy parsing requirements of the legacy Excel format and standardizes column nomenclature. A preliminary data quality assessment revealed several structural anomalies typical of raw Enterprise Resource Planning (ERP) systems. Crucially, the dataset conflated standard purchases with reverse logistics, returns and cancellations denoted by a 'C' in the Invoice number and negative quantities.

3.2 Target Variable

The target variable is the **weekly aggregated sales volume in physical units**, derived from the `Quantity` column. Each observation represents the total net demand for a specific SKU over a standard 7-day Monday-to-Sunday cycle. To stabilize variance across highly skewed volume distributions and to strictly prevent the prediction of negative sales, the target is modeled internally using a logarithmic transformation ($\log(1 + x)$), which is subsequently inversely transformed ($\exp(x) - 1$) for final business evaluation.

The core forecasting task is: given a product's historical sales trajectory, static profiles, and available exogenous information up to week t , predict the consumption for all 12 intervals from $t + 1$ to $t + 12$ (the next fiscal quarter). This 12-week forecasting horizon provides the exact granularity needed for supply chain managers to execute tactical procurement, calibrate safety stock, and optimize warehouse space.

3.3 Predictive Variables

Retail demand forecasting is notoriously complex due to the highly volatile, intermittent, and Pareto-skewed nature of consumer purchasing behavior. A product's sales profile is rarely a

simple mathematical trend; rather, it is a complex superposition of seasonal shopping cycles, price elasticity, historical inertia, and intrinsic product characteristics.

To capture these underlying dynamics and build robust models, we identified and engineered four critical categories of predictive variables. Crucially, the deliberate inclusion of rich exogenous signals, specifically pricing dynamics, promotional flags, and UK holiday schedules, directly addresses the critical univariate limitations of the baseline framework discussed in Section 2.3.

Calendar and Exogenous Holiday Features To capture broad human behavioral cycles and seasonality, `week_of_year`, `month`, and `quarter` are extracted from the transactional timestamps. However, standard linear numeric representations confuse machine learning models at calendar boundaries (e.g., treating December, month 12, as mathematically distant from January, month 1). To preserve the continuous, circular nature of time, we engineered cyclical encodings using Sine and Cosine transformations (`sin_woy`, `cos_woy`, `sin_month`, `cos_month`).

Furthermore, external societal events drastically alter consumption baselines. We utilized the `holidays` library to dynamically compute exact UK National Holidays (`holiday_uk`). Additionally, an explicit `is_christmas_window` flag was engineered to strictly capture the high-volume holiday shopping period spanning from Black Friday (Week 47) through the end of December, a critical feature for isolating annual retail spikes.

Pricing Dynamics and Promotional Elasticity In wholesale and retail, demand is highly elastic to price changes. To account for economic sensitivity, we calculated the `price_weekly` as the median transaction price per SKU per week. To capture market volatility, we engineered `price_percent_change`, reflecting week-over-week pricing momentum.

Most importantly, we introduced a categorical promotion flag (`is_on_promotion`). This heuristic evaluates if the current weekly price drops below 85% of the SKU's training-period historical median price, computed exclusively on weeks prior to the test cutoff to prevent look-ahead leakage. This feature is crucial for isolating non-linear spikes in electricity demand, or rather, sudden bulk orders triggered by heavy discounting, which standard seasonal variables entirely fail to capture.

Autoregressive and Volatility Features To model the strong historical autocorrelation intrinsic to inventory depletion, we generated specific lagged variables: `lag_1` (short-term momentum), `lag_4` (monthly inertia), `lag_13` (quarterly trend), and `lag_52` (exact historical year-over-year baseline).

To smooth out the extreme volatility of daily retail, we engineered moving statistics, specifically `rmean_4`, `rmean_13`, `rstd_4`, and `rstd_13`. Crucially, to strictly prevent data leakage, all rolling statistics are computed on data shifted by one period, meaning the moving average represents the state of the system *prior* to the target week. Finally, we engineered `return_rate_4w` and `return_rate_13w` (the ratio of returned quantities to sold quantities) to flag potential defective batches or customer dissatisfaction cycles that could negatively forecast future demand. Similarly, return rate windows are computed on quantities lagged by one period, ensuring current-week returns are excluded from the feature.

Static Profiling and Categorical Features Unlike a fixed portfolio of clients, retail catalogs suffer from continuous SKU churn, creating severe "Cold-Start" problems for traditional time-series models. To provide cross-sectional context and structural information, we engineered static attributes computed strictly on the training set. First, we calculated a **Demand Profile** for each SKU, incorporating Syntetos-Boylan classification metrics (ADI and CV2) to categorize items into a `demand_class` (smooth, erratic, intermittent, or lumpy) and utilizing `share_zero_weeks` to quantify sparseness. Second, we generated a **Commercial Profile** to capture the general business footprint of the product, including its `price_median`, `mean_basket_size`, `n_unique_customers`, and its domestic market reach (`country_uk_share`).

Finally, to allow the algorithms to leverage broader portfolio similarities, we injected categorical features derived from our unsupervised segmentation: `semantic_cluster_name` (based on 768-dimensional latent text vectors generated via the Google Gemini LLM API) and `volume_tier`. The comprehensive mathematical methodology behind these semantic and volumetric categorizations will be thoroughly detailed in the subsequent Triple-Clustering paragraph 3.4.

3.4 Data Processing & Feature Engineering Pipeline

Once the target and predictive variable requirements were established, the raw 1.06 million-row transactional ledger required extensive preprocessing to be transformed into a machine-learning-ready structure. A multi-step pipeline yields a fully enriched, long-format Parquet dataset, as visualized in Figure 2.

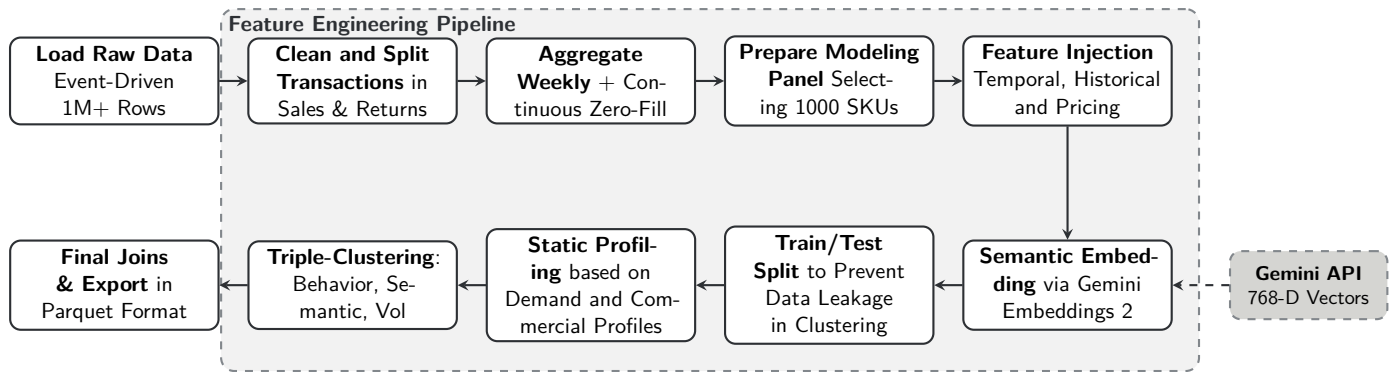


Figure 2: The end-to-end data transformation pipeline. Raw transactions are cleaned of operational noise and split from returns before being aggregated into a continuous weekly panel. The data is enriched with temporal, pricing, and semantic NLP features, isolated via a strict Train/Test split, and segmented through a triple-clustering architecture prior to Parquet export.

Clean and Split Transactions Raw ERP systems often contain administrative artifacts and missing values that heavily distort machine learning algorithms. To ensure data integrity and isolate the true consumer demand signal, the raw ledger was processed through a strict 8-step cleaning and bifurcation pipeline:

1. **Missing Value Removal:** Dropped rows lacking critical transactional information (`InvoiceDate`, `StockCode`, `Description`).

2. **Type Casting:** Cast Invoice and StockCode to strings to prevent the loss of alphanumeric codes during Pandas operations.
3. **Whitespace Stripping:** Removed leading and trailing whitespaces from product descriptions to standardize text for downstream NLP tasks.
4. **Regex Filtering:** Applied a strict Regular Expression (`^\d{5}[A-Za-z]?$`) to isolate genuine product codes, actively removing administrative artifacts like postal fees (POST) and bank charges.
5. **Operational Noise Removal:** Purged warehouse operational notes (e.g., "damaged", "lost", "wet", "check") which represent stock write-offs rather than actual consumer demand.
6. **Price Validation:** Dropped rows with a price less than or equal to zero, eliminating internal accounting adjustments and free samples.
7. **Temporal Alignment:** Generated a Monday-aligned Week column to serve as the foundational chronological index for downstream temporal aggregation.
8. **Bifurcation (Sales vs. Returns):** Separated the dataset based on invoice prefixes and quantities. Invoices starting with 'C' (cancellations) and negative quantities were extracted into a standalone returns dataset, while the remaining positive transactions formed the primary sales dataset.

Crucially, conflating standard purchases with reverse logistics mathematically breaks modern machine learning models. By splitting the data in Step 8, genuine sales form a clean target variable, while returns are preserved to engineer predictive `return_rate` features later in the pipeline. The quantitative impact of this cleaning phase is summarized in Table 4.

Processing Step	Rows Impacted / Removed
Missing Values Removal	4,382
Whitespace Stripping (Modified)	213,035
Invalid StockCode Removal	7,399
Operational Notes Removal	487
Non-Positive Price Removal	1,307
Final Valid Sales (Target Data)	1,035,620
Final Valid Returns (Feature Data)	18,176

Table 4: Data reduction and final bifurcation impact during the transaction cleaning phase.

Aggregate Weekly Retail transaction logs only record events when a sale occurs, resulting in sparse, unevenly spaced chronological data. However, predictive time-series models, and the calculation of autoregressive lags or rolling statistics, strictly require a continuous, evenly spaced temporal index. To bridge this gap, we first grouped the raw transactions, summing both Quantity and Revenue into weekly buckets. Next, we generated a mathematically complete Cartesian grid

spanning all unique SKUs against a continuous W-MON (Weekly-Monday) calendar. The aggregated sales were then reindexed onto this grid. Any week lacking a transaction for a given SKU was explicitly filled with 0. This step guarantees temporal continuity.

Prepare Modeling Panel While continuous zero-filling is mathematically necessary for time-series structures, applying it blindly introduces artificial noise. To resolve this, we implemented a strict three-stage lifecycle trimming and eligibility function:

1. **Leading Zeros (Product Launch):** For each individual SKU, we algorithmically identified its true "launch date" by locating the first chronological week with a recorded Quantity > 0. All artificial zero-filled rows preceding this specific start date were permanently dropped.
2. **Delisted SKUs (Product Churn):** Retail catalogs experience constant product churn. To prevent the models from wasting computational capacity learning to forecast permanently discontinued items, we evaluated SKU activity in the strict 4-week window immediately preceding the global TEST_CUTOFF_DT (thereby preventing data leakage from the test set). Any product registering an aggregate of exactly 0 sales during this final training month was classified as inactive/delisted and was entirely pruned from the modeling pipeline.
3. **High-Value & Long-Term Viability Selection:** Following the core logic of the baseline framework, which restricted modeling to the top 250 revenue-generating items, we significantly expanded the scope to forecast the Top 1,000 SKUs by historical revenue. However, to ensure the forecasting algorithms had a sufficient statistical foundation, these top products required to have a lifetime minimum of 52 active sales weeks, and a recency check of at least 6 active weeks within the final 6 months of training.

These rigorous selection criteria ensure that the models allocate computational capacity exclusively to established, high-impact inventory rather than transient novelty items. The quantitative impact of this lifecycle trimming and filtering phase is summarized in Table 5.

Lifecycle Filtering Step	Quantitative Impact
Leading Zeros Trimmed (Pre-launch)	432,490 rows remaining
Inactive / Delisted SKUs Removed	2,063 SKUs pruned
Final Active Panel (Top 1,000 SKUs)	101,650 rows

Table 5: Data reduction and final panel size after lifecycle trimming and eligibility filtering.

Feature Injection Following the establishment of the continuous, trimmed weekly grid, the dataset was enriched with the dynamic exogenous variables previously detailed in Section 3.3. This pipeline step physically merges the dynamically computed UK national holidays, sine/cosine calendar encodings, historical autoregressive lags, and weekly pricing elasticity metrics directly onto the active modeling panel.

Semantic Embeddings via Gemini API We leveraged Large Language Models for product categorization by extracting the canonical text description for each product and transmitting

them in optimized batches to the Google Gemini-Embedding-2-Preview API. We utilized the CLUSTERING task type to generate 768-dimensional latent vectors. Finally, these vectors were L_2 -normalized to allow for simple Euclidean distance calculations in downstream algorithms.

Train/Test Split Before calculating any global profiles or clusters, the timeline was strictly sliced using a global TEST_CUTOFF_DT. Profiling a product's total yearly volume or seasonal shape using the entire dataset inherently bleeds future information into the historical training features, creating severe Data Leakage. All subsequent static profiles (ADI, CV2) and clustering algorithms were fitted *exclusively* on the training window.

Static Profiling Prior to executing the advanced segmentation algorithms, each SKU was evaluated to establish its baseline behavioral footprint. As thoroughly outlined in the Predictive Variables section, this step computes the Syntetos-Boylan intermittency metrics (ADI, CV2, share_zero_weeks) and the commercial scale indicators (e.g., mean_basket_size, price_median).

Triple-Clustering Rather than forcing a single global model to learn the chaotic and divergent dynamics of 1,000 disparate products, we implemented an unsupervised triple-clustering architecture across three dimensions: seasonal behavior, semantic identity, and commercial scale.

To capture the intrinsic temporal dynamics of each product, we extracted its **52-week average sales profile**. Crucially, before clustering, we applied L_2 -normalization to each SKU's vector. This strips away absolute volume bias, leaving only the pure "shape" of the demand trajectory. We then applied **K-Means** ($k = 5$) to categorize these normalized seasonal profiles.

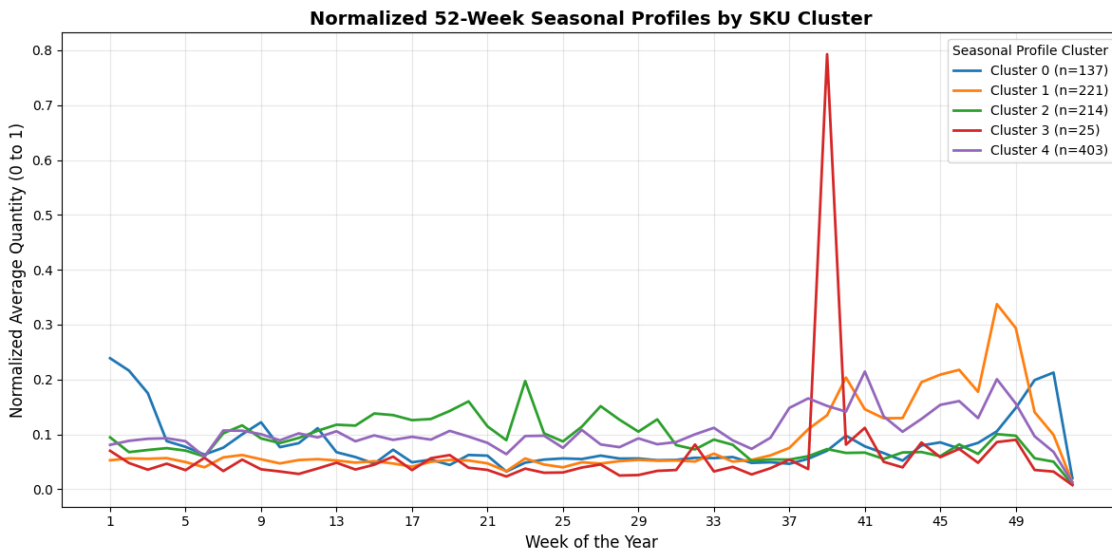


Figure 3: Normalized 52-Week Seasonal Profiles. K-Means clustering ($k = 5$) successfully isolates distinct temporal demand shapes, removing absolute volume bias.

As illustrated in Figure 3, this methodology successfully isolates distinct consumer purchasing rhythms across all five clusters for the top 1,000 SKUs:

- **Cluster 0 (Blue, $n = 137$):** Represents a *New Year & Holiday Boundary Peak*. Demand is highest at the very beginning of the year (Weeks 1-3) and late December (Weeks 50-51), while remaining relatively low and stable during the intervening months.
- **Cluster 1 (Orange, $n = 221$):** Exhibits a pronounced *Pre-Christmas Peak*. Demand remains flat for the first three quarters of the year, before climbing steadily from Week 35 to reach a massive peak around Week 49 to capture late holiday shopping.
- **Cluster 2 (Green, $n = 214$):** Captures a *Spring/Early Summer Peak*. This group maintains a relatively consistent baseline but shows a distinctive, elevated surge during the late spring (peaking sharply around Week 22), before tapering off into the autumn.
- **Cluster 3 (Red, $n = 25$):** Represents a highly specific *Autumn Flash Peak*. This small, niche group of SKUs operates at a near-zero baseline for the entire year except for a massive spike around Week 39, which likely corresponds to specific targeted promotional events.
- **Cluster 4 (Purple, $n = 403$):** Forms the *Stable Base Demand*. As the largest cluster, it represents the core catalog that maintains consistent, steady sales year-round. It shares a similar trajectory to Cluster 1 but with a less pronounced winter lift during Q4.

Time-series models typically struggle with "Cold-Start" products that lack sufficient historical data. To provide structural context, we integrated Large Language Models (LLMs) into the pipeline. Product descriptions were embedded into **768-dimensional latent vectors** using the Google Gemini API. We applied **K-Means** ($k = 15$) directly on these vectors to group items by physical and functional similarity. To make these clusters interpretable for business managers, we applied **Term Frequency-Inverse Document Frequency (TF-IDF)** over the text in each cluster.

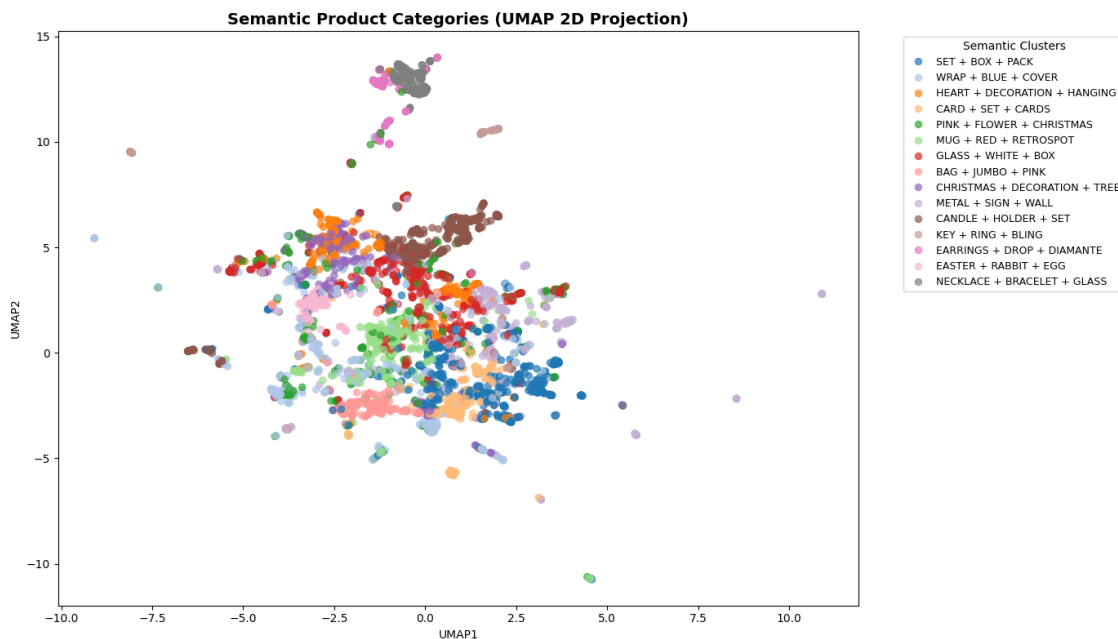


Figure 4: UMAP 2D projection of the 768-dimensional Gemini embeddings. Products are grouped by semantic similarity, with labels generated via TF-IDF.

Figure 4 visualizes the high-dimensional embeddings reduced to a 2D plane via UMAP. The spatial grouping vividly demonstrates the LLM's sophisticated understanding of the retail catalog. It successfully identifies and isolates seasonal holiday themes (e.g., "*CHRISTMAS + DECORATION + TREE*" and "*EASTER + RABBIT + EGG*"), distinct household goods (e.g., "*MUG + RED*" and "*GLASS + WHITE + BOX*"), as well as discrete apparel and accessory categories (e.g., "*EARRINGS + DROP*", "*NECKLACE + BRACELET*").

Retail catalogs are notoriously subject to extreme Pareto skew (the 80/20 rule), where a tiny fraction of SKUs generates the vast majority of volume. Arbitrary segmentation fails to capture the true mathematical distribution of these sales. Instead, we deployed **Jenks Natural Breaks Optimization**, an algorithm designed to minimize intra-class variance and maximize inter-class variance, successfully identifying the natural "cliffs" in product popularity.

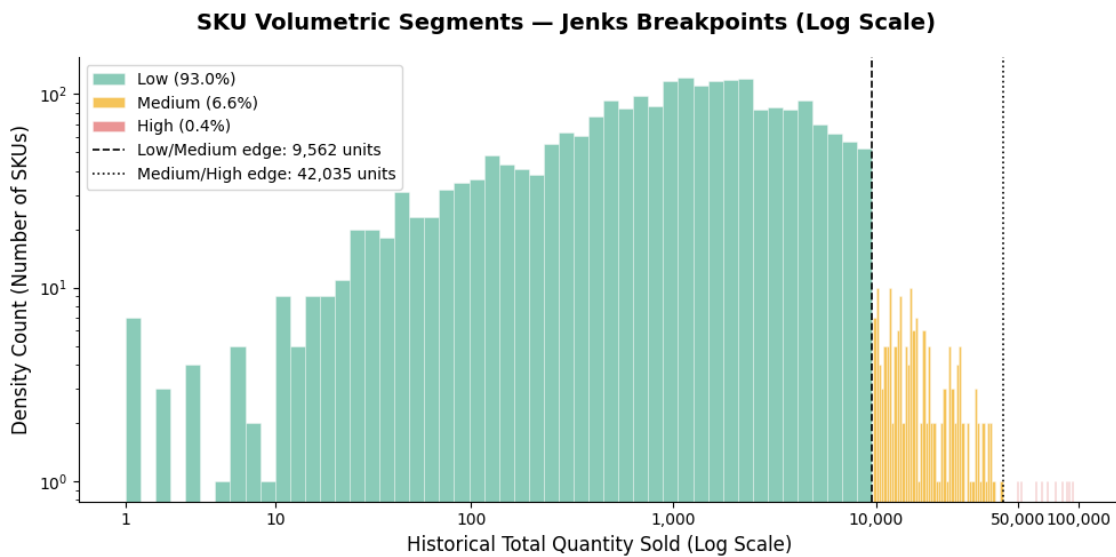


Figure 5: SKU Volumetric Segments derived via Jenks Natural Breaks Optimization. Note the logarithmic scale on the x-axis, highlighting the extreme right-skew of retail volumes.

As shown in Figure 5, the Jenks breakpoints reveal the stark, heavily skewed reality of the retailer's portfolio. The overwhelming majority of the catalog (**93.0%**) falls into the *Low* volume tier, representing items that sell fewer than 9,562 units over their entire lifetime. In stark contrast, a microscopic minority of just **0.4%** of SKUs constitute the *High* tier, the absolute blockbusters moving over 50,000 units (up to 100,000). By injecting this `volume_tier` feature into the model, we effectively prevent the algorithms from forecasting blockbuster-level mathematical spikes for slow-moving, long-tail products, drastically reducing over-forecasting errors on intermittent items.

4 Modeling

4.1 Strategy

The modeling strategy is governed by five foundational principles: chronological validation, cluster-aware training, global hyperparameter tuning, dynamic model selection, and artifact serialization.

Temporal Split & Validation To prevent data leakage, no random shuffling is applied at any stage. The dataset is partitioned strictly chronologically. The out-of-sample testing period is explicitly defined by the global `TEST_CUTOFF_DT` set to "2011-09-01". All transactions subsequent to this date are strictly reserved for evaluating final model performance. To support the automated hyperparameter tuning framework, we implemented a hold-out validation set consisting of the final 12 weeks of the training period. This structure ensures robust hyperparameter optimization.

Cluster-Aware Training Instead of training 1,000 highly noisy individual SKU models or a single overly generalized global model, we employ a Cluster-Aware strategy. Exactly one dedicated forecasting model is trained for each of the five behavioral seasonal clusters previously identified. This approach allows each algorithm to specialize in the unique temporal rhythms of its respective group (e.g., Winter vs. Summer peaks) while drastically reducing computational overhead and benefiting from the variance-reducing effect of aggregated cluster data.

Because retail volumes suffer from massive Pareto skew, predicting raw sales directly leads to extreme bias toward blockbuster items. To evaluate the true business impact, the pipeline implements a *Safe Inverse* procedure: predictions are made in a mathematically compressed space and then inversely transformed back into physical retail units, capped at a physical minimum of 0 units, allowing for a rigorous, volume-weighted portfolio evaluation.

Hyperparameter Tuning To optimize algorithm performance, we implemented an automated hyperparameter tuning framework utilizing Optuna. Crucially, rather than conducting independent hyperparameter searches for each of the five behavioral clusters, the tuning process is executed globally across the entire training dataset. While per-cluster tuning might appear mathematically advantageous, retail demand data inherently includes small, highly volatile segments. Running isolated optimization routines on these noisy segments severely increases the risk of overfitting.

Dynamic Model Selection Recognizing that no single algorithm universally dominates across all demand profiles, the pipeline incorporates an automated SKU-level model selector. After all models generate their out-of-sample predictions, this evaluation function calculates the Weighted MAPE independently for every single SKU. It then constructs a definitive mapping that assigns the highest-performing forecasting architecture to each specific product.

MLOps Artifact Serialization The modeling phase serves as the backend engine for the Agentic AI Assistant. Upon the successful completion of a training cycle, each model pipeline serializes its state into the `agent/artifacts/` directory. These artifacts include the global feature scalers, the exact list of expected exogenous variables, the SKU-to-Cluster mapping dictionaries, the optimal SKU-to-Model routing dictionary, and the fitted model objects.

4.2 Ridge Regression

To establish a robust baseline of how well standard autoregressive features, pricing dynamics, and categorical profiles can predict future retail consumption, we implement a Multiple Linear Regression model with L2 regularization (Ridge). This baseline serves to quantify the linear relationships between historical lags and future demand, acting as the benchmark against which more complex non-linear algorithms are evaluated.

4.2.1 Scaling and Feature Engineering

Because retail sales volumes span several orders of magnitude (from dozens of units to tens of thousands), an unscaled linear model would suffer from severe heteroscedasticity and weight explosion. To ensure numerical stability, the feature engineering and scaling pipeline executes the following steps before training:

1. **Categorical Encoding:** Static profile attributes (`volume_tier`, `semantic_cluster_name`, and `demand_class`) are one-hot encoded to allow the linear model to apply fixed coefficient biases based on product identity and scale.
2. **Logarithmic Transformation for Volumes:** The target variable (`Quantity`), alongside all autoregressive volume features (`lag_1` through `lag_52`) and rolling standard deviation-s/means, are transformed natively using `np.log1p(x)`. This stabilizes the massive variance and scales extreme blockbuster spikes down to a manageable linear space.
3. **MinMax Scaling for Ratios:** Ratio-based and static numerical features (e.g., `return_rate_4w`, `price_percent_change`, `ADI`, `CV2`) are globally normalized between 0 and 1 using a standard `MinMaxScaler`.
4. **Regularization:** A Ridge regressor is utilized rather than Ordinary Least Squares (OLS) to penalize excessive coefficient weights and mitigate the severe multicollinearity inherent in highly correlated autoregressive lags. The regularization strength (α) is optimized globally through the Optuna framework. The optimization identified an optimal $\alpha \approx 99.32$.

4.2.2 Inference and Evaluation

During the out-of-sample testing phase, predictions are generated using a vectorized, cluster-level approach, applying the globally-tuned α parameter across all behavioral segments. Because the linear models strictly map the pre-calculated exogenous and autoregressive features to the target, the inference engine executes `model.predict()` simultaneously across all rows belonging to a specific seasonal cluster. Following cluster-level prediction, the log-scaled outputs are then inversely transformed using `np.expm1(x)` to convert them back into physical retail units. Finally, predictions are strictly bounded at 0 units to enforce physical constraints (a product cannot sell negative units).

For comprehensive portfolio evaluation, we delegate the performance calculation to our centralized `evaluation.py` module. By aggregating the total absolute error divided by the total actual consumption, the WMAPE metric naturally resolves the zero-bound problem inherent in intermittent retail data, strictly weighting the forecasting errors by the SKU's true physical volume.



Figure 6: Ridge Regression baseline forecasts versus actual retail consumption. The predictions capture basic seasonal trends and autoregressive inertia but may struggle with highly non-linear promotional spikes.

4.3 Prophet

To capture the complex, overlapping seasonalities inherent in long-term retail demand, we implement the Facebook Prophet algorithm. Prophet's internal use of Fourier series is highly efficient for modeling continuous annual cycles without the high dimensionality of one-hot encoded variables. However, because Fourier series generate inherently smooth and continuous curves, the model struggles to capture the sharp, non-linear spikes caused by isolated promotional events or sudden stock-outs.

4.3.1 Model Configuration and Feature Engineering

Prophet requires a strict internal data schema: the temporal index must be mapped to a `ds` column, and the target must be mapped to `y`. The feature preparation and model configuration diverge significantly from the baseline autoregressive approach:

1. **Target Transformation & Aggregation:** To stabilize variance across intermittent SKUs, the physical Quantity is transformed using `np.log1p()` prior to training. Furthermore, because Prophet trains a single generalized curve per cluster, both the log-scaled target and the continuous exogenous regressors are aggregated (averaged) by `profile_cluster_id` and `Week`.
2. **Seasonality & Trend Optimization:** We configure the model to explicitly capture macroeconomic retail cycles with `yearly_seasonality=True`. To prevent underfitting or extreme variance, the trend flexibility (`changepoint_prior_scale`) is optimized globally via the Optuna framework. The optimization identified an optimal scale of ≈ 0.114 . United Kingdom national holidays are natively integrated, allowing the model to explicitly learn and isolate the "holiday effect".
3. **Exogenous Retail Regressors:** To anchor Prophet's smooth curves to real-world commercial volatility, we explicitly register external regressors via the `add_regressor()` method. We inject `is_christmas_window`, `is_on_promotion`, and `price_percent_change` to account for non-linear commercial shocks, alongside `lag_1` and `lag_4` to maintain short-term operational inertia.

4.3.2 Inference and Evaluation

During the out-of-sample testing phase, forecasts are generated for each cluster. The model builds a future dataframe spanning the 12-week horizon containing the known temporal index and the specific exogenous regressors, executing a vectorized prediction via `m.predict()`.

The cluster-level pattern forecasts are then merged back onto the individual SKUs via a Left Join. To evaluate the physical business impact, the dimensionless log-predictions are bounded to a mathematical ceiling (to prevent exponential overflow) and inversely transformed using `np.expm1(x)`. To enforce physical reality, any negative predictions are clipped to exactly 0 units.

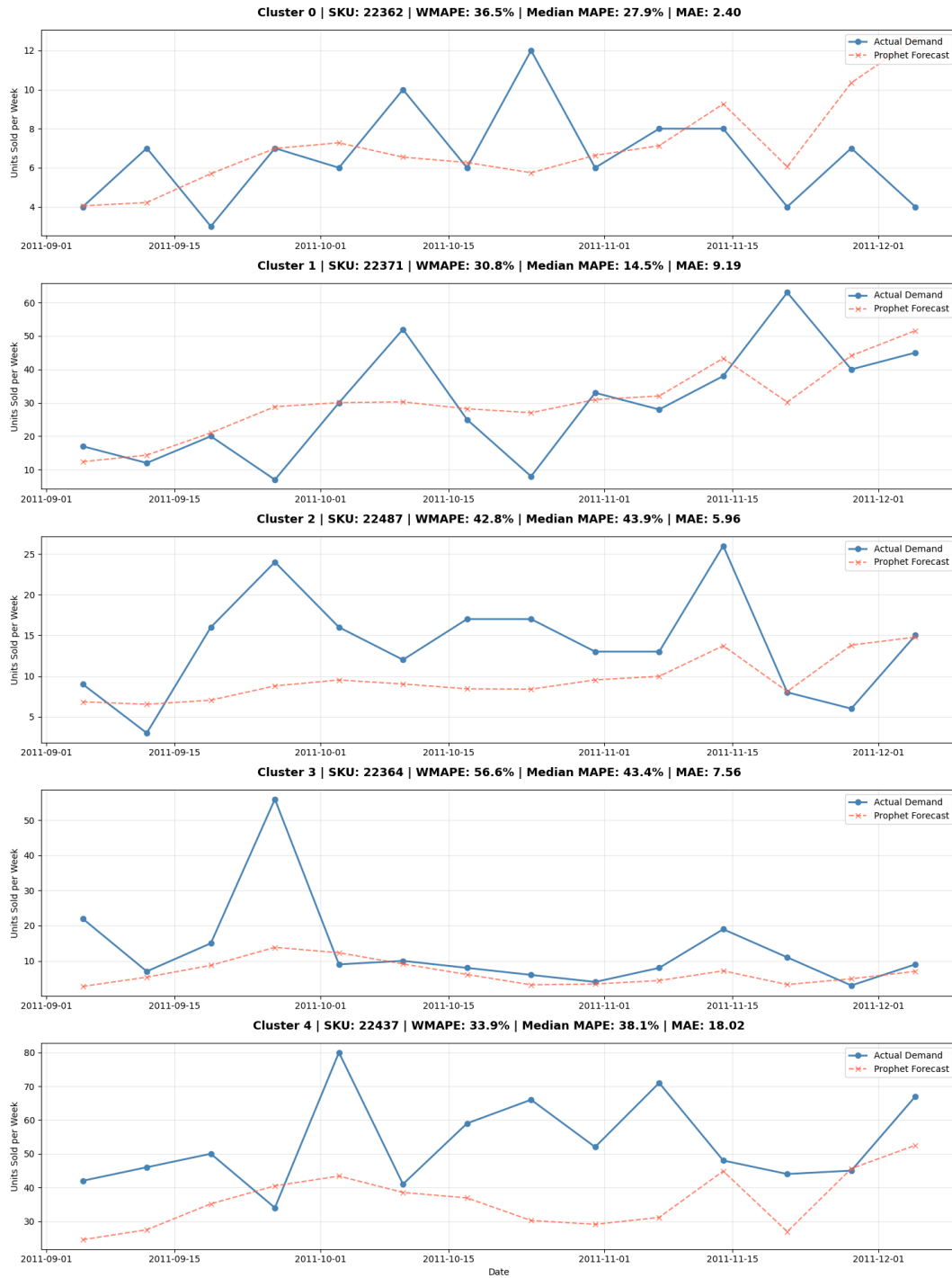


Figure 7: Prophet: Forecast vs. Actual Retail Consumption. The inclusion of pricing regressors and specific historical lags helps the algorithm track sudden promotional peaks that standard Fourier curves would miss.

4.4 LightGBM

While linear models provide stability and Prophet excels at macro-seasonality, retail catalogs are notoriously non-linear. The interaction between semantic product identity, localized price drops, and historical inertia is highly complex. To capture these multi-dimensional interactions without assuming a linear relationship, we implement a LightGBM Regressor.

4.4.1 Native Categorical Handling and The Tweedie Objective

LightGBM operates fundamentally differently from the Ridge baseline and Prophet, leading to two major architectural advantages for our retail pipeline:

1. **Native Categorical Support:** Unlike the Ridge regression pipeline which required computationally heavy `pd.get_dummies` expansion, LightGBM handles categorical variables (`volume_tier`, `semantic_cluster_name`, and `demand_class`) internally. By utilizing a highly optimized histogram-based split finding algorithm, it evaluates categorical splits naturally without inflating the feature space.
2. **The Tweedie Distribution:** Retail demand is heavily "zero-inflated" (intermittent) and right-skewed (long-tail). Standard loss functions like Mean Squared Error (MSE) heavily penalize outliers and struggle with predicting exact zeros. We configured the LightGBM regressor with `objective="tweedie"`. Rather than hardcoding the distribution parameters, we utilized Optuna to globally optimize the `tweedie_variance_power`, which converged on an optimal value of ≈ 1.77 . The Tweedie distribution acts as a mathematical bridge between the Poisson distribution (ideal for discrete counts/zeros) and the Gamma distribution (ideal for continuous positive values), making it the absolute state-of-the-art objective function for intermittent retail forecasting.

To ensure a fair benchmark, the LightGBM model ingests the exact same globally scaled features (Log1p for volumes, MinMax for ratios) utilized by the Ridge baseline.

4.4.2 Training and Inference

We maintain the Cluster-Aware architecture, training a dedicated LightGBM ensemble for each seasonal profile cluster using the globally optimized hyperparameters. Through 50 Optuna trials evaluated on the 12-week hold-out set, the optimal tree configuration was identified (`n_estimators=403`, `learning_rate` ≈ 0.023 , `num_leaves=28`, `min_data_in_leaf=11`), augmented by L1/L2 regularization (`reg_alpha` ≈ 0.208 , `reg_lambda` ≈ 0.004). The combination of structural leaf constraints and L1/L2 penalties actively prevents the algorithm from building ultra-specific rules that overfit to a single SKU's anomalous history.

During the 12-week inference window, the models predict the log-scaled targets, which are subsequently merged, inverse-scaled via `expm1`, and bounded to zero. Because tree-based models make no assumptions about the continuity of the data, LightGBM acts as our most aggressive model, capable of predicting sudden vertical spikes in consumption (e.g., triggered by a sudden activation of `is_on_promotion`) and dropping immediately back to absolute zero the following week.

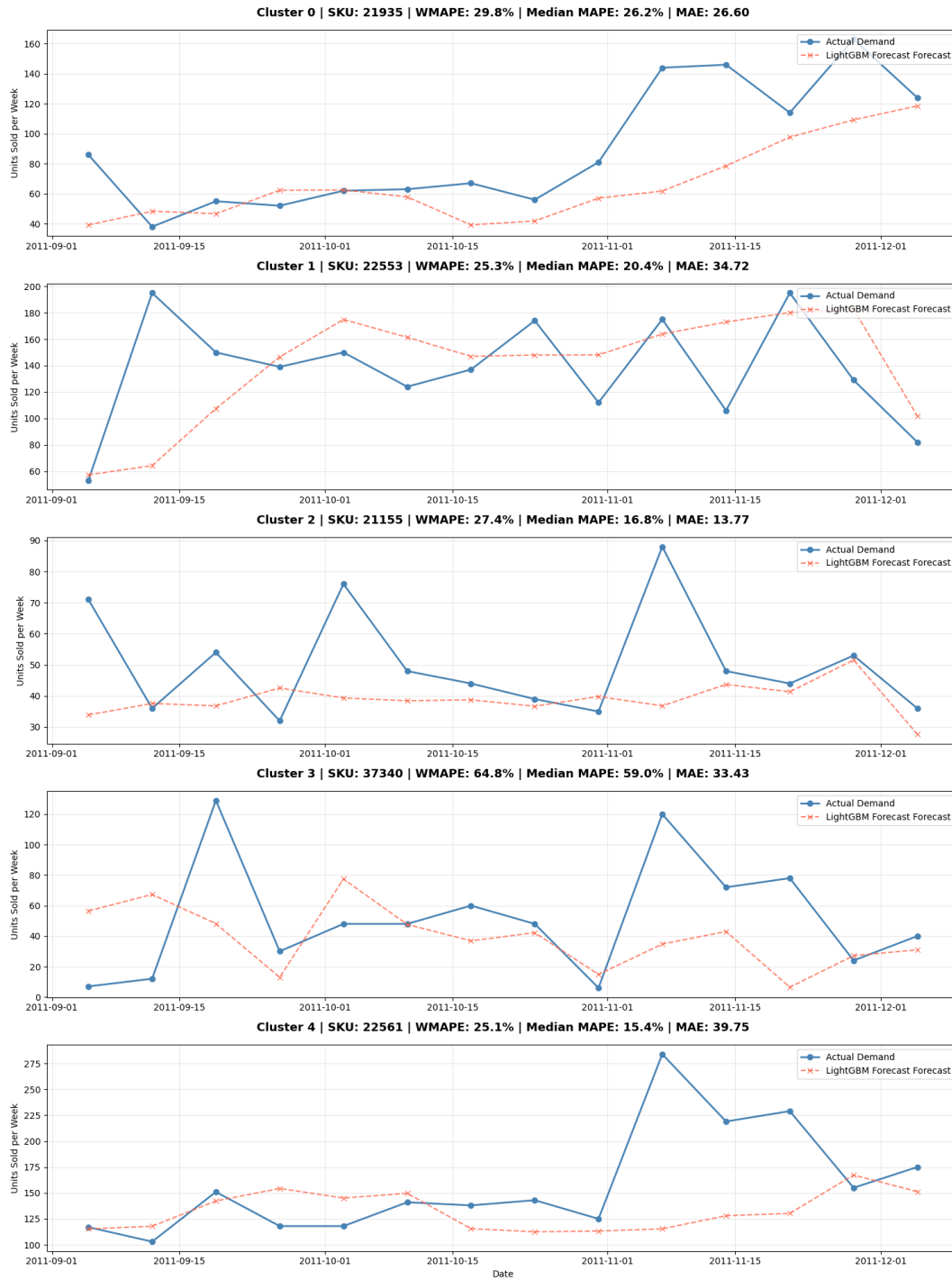


Figure 8: LightGBM: Forecast vs. Actual Retail Consumption. By leveraging the Tweedie objective and non-linear split-finding, LightGBM effectively captures intermittent demand patterns and sudden promotional spikes more aggressively than linear or curve-fitting baselines.

4.5 SARIMAX (Seasonal ARIMA with eXogenous Variables)

While LightGBM excels at non-linear interactions and Prophet models macro-seasonality via continuous Fourier curves, classical econometric models remain the gold standard for understanding exact historical memory. To capture both short-term demand shocks and exact year-over-year seasonal cycles, we implement a SARIMAX architecture.

4.5.1 Model Configuration and Hyperparameter Tuning

Because SARIMAX models the mathematical differences between time steps, it requires a strictly chronological and continuous time series. The feature preparation and configuration follow a cluster-aggregated approach:

1. **Target Transformation & Aggregation:** To stabilize the variance of highly skewed retail sales and satisfy the statistical assumptions of ARIMA models, the Quantity is transformed natively using `np.log1p()`. To prevent the model from failing to converge on the extreme sparsity of individual SKUs, the target and exogenous variables are aggregated by `profile_cluster_id` and sorted chronologically.
2. **Exogenous Regressors:** To anchor the autoregressive math to real-world commercial realities, we pass the dynamically scaled exogenous features (`is_christmas_window`, `is_on_promotion`, `price_percent_change`, `lag_1`, `lag_4`) directly into the model's `exog` parameter.
3. **Global Optimization via Optuna:** SARIMAX requires defining the order for both local dynamics (p, d, q) and seasonal memory (P, D, Q, s). The Optuna algorithm converged on a highly specific configuration: Order (0, 2, 1) and Seasonal Order (1, 0, 1, 52). This optimized configuration reveals profound insights into the portfolio's behavior:
 - **Local Dynamics (0, 2, 1):** The double differencing ($d = 2$) is a significant finding; it indicates that the baseline trend is not just linear, but subject to non-linear acceleration or deceleration. By ignoring standard autoregression ($p = 0$) and relying heavily on the moving average of past errors ($q = 1$), the model acts highly reactively, correcting its trajectory based exclusively on recent forecast mistakes rather than absolute past sales.
 - **Seasonal Memory (1, 0, 1, 52):** The 52-week seasonal period ($s = 52$) locks the model to the annual retail cycle. The configuration ($P = 1, Q = 1$) ensures the model looks back exactly one year to retrieve both the historical baseline demand and the historical error rate, applying a double-layered seasonal correction.

4.5.2 Inference, Evaluation, and Serialization

During the out-of-sample testing phase, the fitted SARIMAX models generate a 12-week forecast utilizing the `forecast()` method alongside the future exogenous matrix (`X_test`).

Following the cluster-level projections, predictions are merged back to the individual SKUs. The dimensionless log-predictions are bounded at an upper ceiling to prevent exponential overflow, inversely transformed via `np.expm1()`, and rigorously clipped to 0 to prevent the prediction of negative retail units.

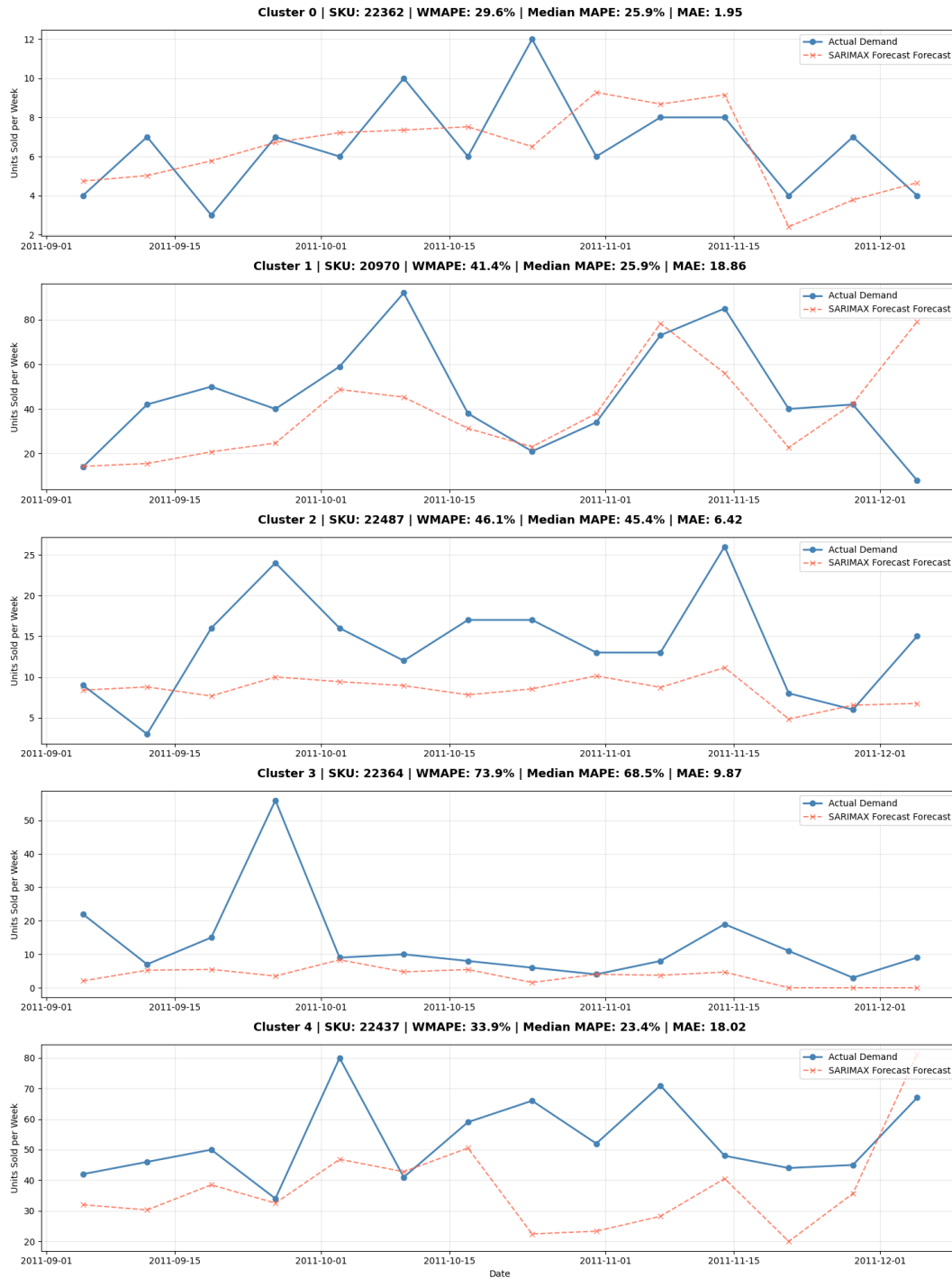


Figure 9: SARIMAX: Forecast vs. Actual Retail Consumption. By leveraging explicit 52-week seasonal memory and moving-average error correction, the model excels at replicating exact historical patterns, though it may be more computationally rigid than tree-based alternatives.

4.6 Non-Stationary Transformer

While linear models and tree-based algorithms like LightGBM provide robust baselines, retail demand is fundamentally driven by complex, overlapping temporal cycles and volatile commercial shifts. To capture these deep sequential dependencies, we implemented the **Non-Stationary Transformer**, based on the groundbreaking architecture proposed by Liu et al. (NeurIPS 2022).

Standard Transformers, despite their success in natural language, often suffer severe performance degradation on real-world time series. This occurs because the joint distribution of real-world data changes over time (non-stationarity). Traditional approaches attempt to fix this by standardizing the inputs; however, this causes *over-stationarization*, destroying the intrinsic variance of the data and forcing the Transformer to generate indistinguishable, uniform attention maps (see Figure 10).

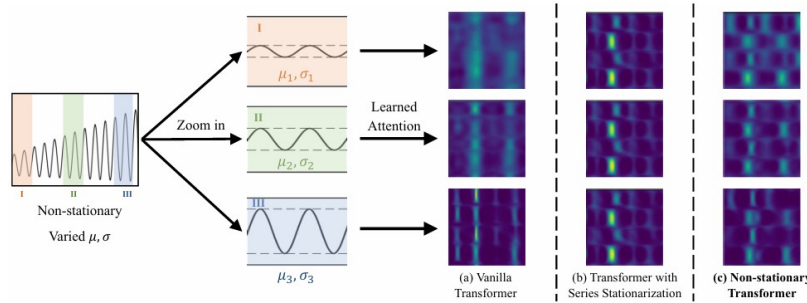


Figure 10: The Over-Stationarization Problem: standard normalization causes conventional Transformers to lose the capacity to generate distinguishable temporal attentions across different series.

4.6.1 PyTorch Architecture and De-Stationary Attention

To solve this, our PyTorch implementation introduces two native modules that wrap the core Encoder-Decoder architecture:

1. **Series Stationarization Module:** Before entering the network, each input window is locally translated and scaled to attenuate massive non-stationarity:

$$x'_i = \frac{x_i - \mu_x}{\sigma_x}$$

A symmetrical *De-normalization Module* is applied at the very end of the network to project the base model's predictions back into the original scale:

$$\hat{y}_i = \sigma_x \odot y'_i + \mu_x$$

2. **De-Stationary Attention:** To prevent the self-attention mechanism from going blind due to the normalized inputs, the architecture utilizes two Multi-Layer Perceptrons (Projectors) to learn scaling (τ) and shifting (Δ) factors directly from the un-stationarized data's statistics. These factors are dynamically injected into the attention calculation:

$$\log \tau = MLP(\sigma_x, x), \quad \Delta = MLP(\mu_x, x)$$

$$\text{Attn}(Q', K', V', \tau, \Delta) = \text{Softmax} \left(\frac{\tau Q' K'^T + \mathbf{1} \Delta^T}{\sqrt{d_k}} \right) V'$$

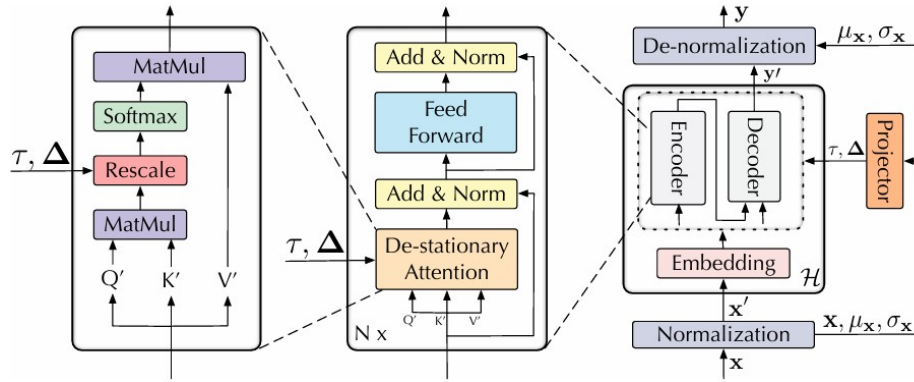


Figure 11: The Non-Stationary Transformer Architecture, integrating the Series Stationarization mechanism and the Projector MLPs for De-Stationary Attention.

4.6.2 Data Engineering and Tensor Construction

Implementing a Deep Learning model on a highly skewed, intermittent retail dataset spanning 1,000 top SKUs poses severe computational and optimization challenges. To stabilize training, we apply our **Cluster-Aware Strategy**, training one Transformer per seasonal behavioral cluster.

To address the extreme variance of retail volumes (Pareto skew), the raw target quantities are natively compressed using a `np.log1p()` transformation prior to tensor construction. We utilize a sliding window dataset generator mapping the continuous weekly timelines into fixed tensors: the Encoder receives the past 16 weeks of context (`SEQ_LEN = 16`), while the Decoder receives a 12-week historical overlap (`LABEL_LEN`) and outputs the final 12-week forecast horizon (`PRED_LEN`). Time features, including normalized week-of-year, sine/cosine cyclical encodings, and month representations, are extracted to serve as dense temporal embeddings (`enc_mark`, `dec_mark`). Finally, to combat overfitting on small clusters, a Gaussian noise augmentation (`AUG_NOISE_STD`) is injected dynamically during training.

4.6.3 Inference, Inverse Scaling, and MLOps Artifacts

During the out-of-sample testing phase, the model executes a vectorized 12-week forecast. Because the Transformer operates strictly in the log-scaled space, the raw predictions must be translated back to physical reality. To prevent mathematical hallucinations, where a minor error in log-space exponentially explodes into millions of phantom units, the network's output is capped at a strict upper boundary before being inversely transformed via `np.expm1()`. Predictions are subsequently bounded at 0 units to enforce physical retail constraints.

Consistent with our MLOps philosophy, the trained architecture is serialized for the Agentic AI Assistant. Crucially, rather than pickling the entire complex PyTorch object, which frequently causes deployment crashes across different environments, we extract and save the raw tensor weights

via `model.state_dict()`. This guarantees that the Agentic layer can flawlessly instantiate the Transformer and run real-time *De-Stationary Attention* queries in production.

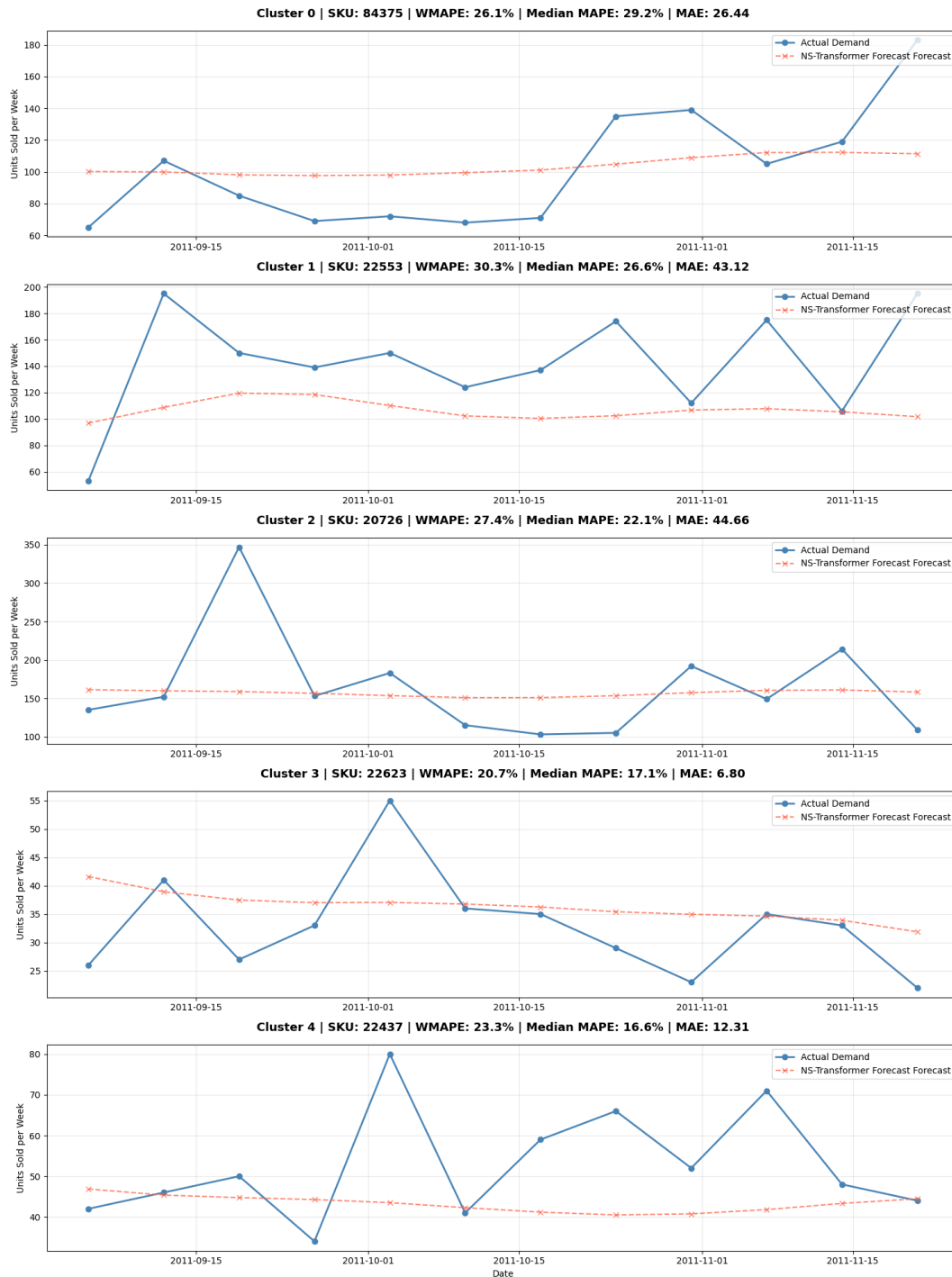


Figure 12: Non-Stationary Transformer: Forecast vs. Actuals. By dynamically adjusting attention weights via the De-Stationary mechanism, the network successfully maps volatile seasonal peaks without requiring strict linear autoregression.

5 The Agentic Layer: AI Forecasting Assistant

While highly accurate machine learning pipelines are the core of modern retail forecasting, their ultimate value is realized only when business stakeholders can interact with them frictionlessly. To bridge the gap between complex MLOps architectures and executive decision-making, we designed and deployed an **Agentic AI Assistant** using the LangGraph framework and large language models (supporting OpenAI, Gemini, Claude, and Ollama).

5.1 Agent Architecture and The ReAct Framework

The assistant is built upon the **ReAct (Reasoning and Acting)** paradigm. Instead of relying purely on its pre-trained parametric memory, which lacks awareness of our proprietary retail catalog, the LLM is equipped with specialized Python tools. When a user submits a query, the Agent initiates a reasoning loop: it parses the intent, selects the appropriate analytical tool, executes the Python code against our local MLOps backend, observes the output, and finally synthesizes a human-readable response.

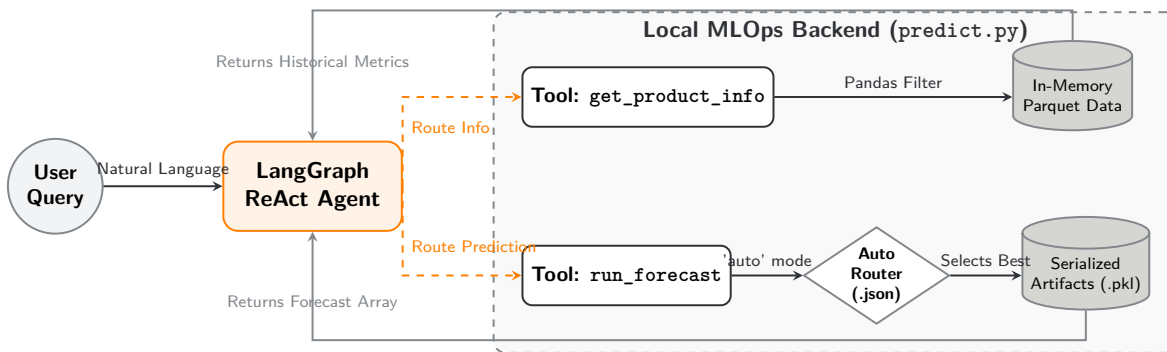


Figure 13: The Agentic Workflow. The LLM acts as a router, mapping natural language requests to specific Python tools. In forecasting, the backend utilizes an Auto-Router to dynamically select the mathematically optimal model per SKU before loading the serialized artifacts.

5.2 Tool Integration and Context Retrieval

To empower the Agent, we injected two primary capabilities into its execution environment:

- **get_product_info(stock_code)**: Queries the master Parquet dataset loaded in RAM to retrieve a product's historical mean, peak sales, total lifetime volume, semantic category, and behavioral shape (e.g., dynamically translating Cluster 1 to "Winter/Christmas Peak"). This operates in sub-second latency, providing the LLM with immediate contextual grounding.
- **run_forecast(stock_code, model, horizon_weeks)**: Triggers the predict.py inference engine. To maximize analytical value and minimize human error, the Agent's system prompt strictly mandates an *Auto-Routing Strategy*: unless explicitly overridden by the user, the Agent invokes this tool in 'auto' mode. The backend engine reads the **best_model_per_sku.json** routing dictionary (compiled during the model evaluation phase) to uniquely identify the mathematically optimal algorithm for the requested product.

The `predict.py` module serves as the mathematical core of the agentic layer. Based on the Auto-Router's decision, it dynamically loads the corresponding `.pkl` artifacts, whether LightGBM, Ridge, Prophet, SARIMAX, or the NS-Transformer, from the `agent/artifacts/` directory. Crucially, the engine handles the on-the-fly execution of the Log1p and MinMax scaling, categorizing variables exactly as required by the specific model's architecture, and safely performing the `expm1` inversion before returning the physical units to the LLM.

A critical UX challenge in human-AI interaction is input volatility. Users frequently utilize shorthand (e.g., requesting product "85123a" in lowercase or with trailing spaces). Custom Python preprocessing logic automatically intercepts, strips, and uppercases the user's string prior to execution, ensuring bulletproof tool reliability against the strict database indexes.

5.3 Prompt Engineering and Business Interpretation

Raw numerical output, while mathematically correct, is often insufficient for business stakeholders. The System Prompt driving the LangGraph Agent enforces a strict response hierarchy. The Agent must format its output into an Executive Summary, followed by Markdown-formatted performance metrics (Total Sales Volume, Average Weekly Units, Peak Weekly Units), and conclude with an **Analyst Interpretation**.

Because the Agent has access to both the cluster's behavioral profile (via the `info` tool) and the explicit knowledge of which forecasting algorithm was automatically selected, it leverages its pre-trained reasoning to infer commercial characteristics. For instance, if the backend auto-selects SARIMAX and predicts a sharp peak around Week 49 for a "Winter/Christmas Peak" item, the LLM autonomously flags this context, informs the user that an autoregressive econometric model was chosen due to its strong seasonal memory, and suggests potential supply chain implications, such as early warehouse staging or optimized procurement planning.

5.4 Example Interaction

The resulting synergy between the robust MLOps backend and the conversational frontend yields a highly professional, instant analytical asset, as demonstrated in the following terminal interaction:

```
federicogiorgi@Federicos-MacBook-Air forecasting-retail % python agent/chatbot.py
```

```
Initializing LLM Agent using provider: GEMINI
```

```
Retail Demand Analyst
```

```
1000 products mapped to clusters · type 'exit' to quit
```

```
You: Can you provide the historical profile and sales summary for product 22423? I want to know its seasonal behavior and volume tier.
```

```
Agent:
```

```
Here is the historical profile and sales summary for product 22423:
```

- **Semantic Category:** GLASS + WHITE + BOX
- **Volume Tier:** Medium
- **Seasonal Profile:** Winter/Christmas Peak (Cluster 1)

Analyst Interpretation

This means that product 22423 experiences its highest demand during the winter months, particularly around the Christmas period. For inventory management, this implies that stock levels should be significantly increased in anticipation of this peak season to avoid stockouts, and then gradually reduced post-holiday.

- **Historical Average Sales:** 303.0 units/week
- **Historical Peak Sales:** 1551 units/week
- **Total Sales Volume (Lifetime):** 27,577 units

You: Forecast the demand for product 85123A for the next 8 weeks. Please use the 'auto' mode to select the most accurate model for this specific item.

Agent:

I have generated a demand forecast for product 85123A for the next 8 weeks using the 'auto' model selection, which chose **LightGBM (LGB)** as the most accurate model for this product. Here's a summary of the forecast:

- **Forecast Period:** September 5, 2011, to October 24, 2011
- **Total Sales Forecast:** 4,410 units
- **Average Weekly Sales Forecast:** 551.3 units/week
- **Peak Weekly Sales Forecast:** 649.0 units (expected in the week of October 10, 2011)

6 Results & Evaluation

The evaluation of the forecasting engine is conducted through a strict Business-Oriented lens. Rather than treating the 1,000 SKUs as abstract mathematical arrays, we evaluate the models based on their capacity to optimize inventory, mitigate stock-out risks, and prevent over-purchasing. To achieve this, we utilize a combination of specific metrics: *WMAPE* for volume-weighted portfolio assessment, *Median MAPE* for typical item-level accuracy, and *MAE* for physical unit deviations.

6.1 Evaluation Metrics: A Retail Supply Chain Perspective

Standard statistical metrics often fail to capture the commercial reality of retail sales. Our evaluation strategy is designed to explicitly address two inherent industry challenges: the "Intermittent Demand" noise (frequent zero-sales weeks) and the extreme "Pareto Volume" disparity (blockbusters vs. long-tail items).

WMAPE (Weighted Mean Absolute Percentage Error) The standard MAPE is heavily asymmetric and mathematically undefined when actual sales are zero, a frequent occurrence in retail. More importantly, it is "volume-blind." A 100% error on a slow-moving item (e.g., predicting 2 units instead of 1) is financially negligible compared to a 10% error on a blockbuster item (e.g., predicting 4,500 units instead of 5,000). Since procurement budgets and warehouse capacities are dictated by absolute volume, we utilize the WMAPE as our primary economic benchmark:

$$\text{WMAPE} = \frac{\sum_{t=1}^T \sum_{i=1}^N |y_{i,t} - \hat{y}_{i,t}|}{\sum_{t=1}^T \sum_{i=1}^N y_{i,t}} \times 100$$

where $y_{i,t}$ represents the actual sold quantity of SKU i at week t . Crucially, WMAPE pools the absolute error and divides it by the total actual volume. This naturally resolves the division-by-zero problem and ensures that high-volume SKUs, the ones that matter most for inventory logistics, correctly dominate the performance score.

Median MAPE & MAE WMAPE can mask poor algorithmic performance on the long-tail of the catalog. To diagnose "shape accuracy" at the individual product level, we report the **Median MAPE**. By isolating only the active sales weeks ($y > 0$) and computing the median of the absolute percentage errors across the cluster, we effectively filter out the mathematical noise of zero-demand periods and extreme percentage outliers.

Furthermore, we compute the **Mean Absolute Error (MAE)** to provide business stakeholders with a highly interpretable metric expressed in actual physical retail units (e.g., indicating that a forecast is, on average, "off by 4 units per week").

Global vs. Cluster Granularity Our evaluation engine provides a dual-layered view. We report metrics at the *Global* level to inform executive-level supply chain planning, and at the *Cluster* level to identify specific behavioral segments (e.g., Winter Peaks vs. Stable Demand) where the predictive engine excels or requires further refinement.

6.2 Model Performance & Benchmarking

All models produce per-cluster breakdowns utilizing the K-Means assignments. This allows us to identify which behavioral segments are inherently harder to forecast and to attribute metric differences to structural algorithmic causes rather than SKU-level noise.

Model Architecture	Global WMAPE	Global Median MAPE	Global MAE
LightGBM (Tweedie)	57.13%	52.46%	41.44
NS-Transformer (SOTA)	60.81%	55.86%	43.53
Ridge Regression (Baseline)	60.99%	56.54%	44.24
Facebook Prophet	89.37%	78.46%	64.82
SARIMAX	90.28%	79.61%	65.49

Table 6: Global Model Performance Comparison (12-Week Out-of-Sample Test)

The updated results reveal critical architectural insights regarding retail demand forecasting:

- **The Dominance of Tree-Based Models:** LightGBM emerges as the undisputed global leader (57.13% WMAPE). Its native handling of categorical features and, most importantly, its use of the Tweedie objective function allows it to mathematically conquer the intermittent, zero-inflated nature of the retail catalog.
- **Deep Learning vs. Linear Baselines:** The Non-Stationary Transformer (60.81%) and Ridge Regression (60.99%) perform almost identically at the global level. Ridge benefits from strict autoregressive anchoring, while the Transformer leverages its De-Stationary Attention to map complex seasonal patterns without rigid linear constraints.
- **The Failure of Continuous Curves:** Prophet (89.37%) and SARIMAX (90.28%) mathematically collapse when applied to this dataset. Retail data is simply too sparse and discontinuous. Prophet's smooth Fourier series and SARIMAX's rigid autoregressive matrices cannot accurately model a product that sells 0 units for five weeks and then violently spikes to 500 units during a localized promotion.

ID	Behavioral Profile	Size	WMAPE by Architecture			
			Ridge	LightGBM	NST	SARIMAX
0	New Year & Holiday Boundary	137	69.29%	61.61%	81.25%	94.72%
1	Pre-Christmas Peak	221	62.31%	58.82%	68.45%	91.88%
2	Spring/Early Summer Peak	214	64.24%	59.84%	58.24%	93.25%
3	Autumn Flash Peak	25	69.31%	146.80%	66.12%	91.66%
4	Stable Base Demand	403	58.28%	53.50%	54.11%	87.98%

Table 7: WMAPE Performance Breakdown by Behavioral Cluster. (Prophet is excluded for brevity due to uniform underperformance across all segments).

The Cluster 3 Anomaly and Auto-Routing Validation The cluster breakdown exposes a fascinating vulnerability in our highest-performing model. While LightGBM easily wins on stable or traditionally seasonal clusters (Clusters 1 and 4), it suffers a catastrophic mathematical failure on Cluster 3 (Autumn Flash Peak), exploding to a 146.80% WMAPE. Because Cluster 3 contains only 25 highly erratic SKUs that remain at zero before violently spiking in Week 39, the gradient boosting trees completely misinterpret the timing and magnitude of the anomaly.

In stark contrast, the Non-Stationary Transformer and Ridge Regression handle this exact same volatile cluster with relative ease (66.12% and 69.31% respectively). This precise insight mathematically validates our **Dynamic Model Selection** strategy implemented in the Agentic Layer: the 'auto' mode will autonomously route these highly volatile, spiky SKUs to the Transformer or Ridge algorithms, while routing the vast majority of the stable catalog to LightGBM, ultimately minimizing systemic risk.

6.3 Temporal Error Distribution

Retail demand is highly dynamic, particularly during the transition from the relatively stable late-summer baseline into the extreme volatility of the Q4 holiday shopping season (Black Friday and Christmas). To ensure the forecasting models are structurally robust across this transition, we evaluated the WMAPE across four distinct, sequential time bins within the 12-week out-of-sample test horizon (September to December 2011).

Testing Period (Q4 2011)	WMAPE by Forecasting Architecture			
	Ridge (Baseline)	LightGBM	NS-Transformer	SARIMAX
Period 1: Early Sep – Late Sep	60.63%	57.30%	67.28%	89.99%
Period 2: Late Sep – Late Oct	62.70%	60.13%	64.21%	89.55%
Period 3: Late Oct – Mid Nov	60.48%	56.49%	58.44%	90.43%
Period 4: Mid Nov – Early Dec	60.51%	55.46%	54.68%	90.89%

Table 8: Temporal Evolution of WMAPE across the 12-Week Testing Horizon. (Prophet mirrors SARIMAX's uniform decay and is excluded for brevity).

Horizon Resilience and The LightGBM Baseline The temporal analysis highlights a fundamental difference in how machine learning models digest the holiday ramp-up. LightGBM demonstrates exceptional temporal stability. Despite the increasing volatility of consumer behavior approaching November and December, its volume-weighted error actually *decreases* (from 57.30% in Period 1 down to 55.46% in Period 4). This proves that the algorithm successfully learned the non-linear logic of promotional flags (`is_on_promotion`) and holiday schedules, accurately anticipating the Q4 volume surges. Similarly, the Ridge Regression baseline maintains a strict, flat error rate (fluctuating by a mere $\sim 2\%$), reliably anchoring its predictions to recent autoregressive inertia.

The Transformer's Peak Specialization The most fascinating architectural behavior is observed in the Non-Stationary Transformer (NST). Unlike the tree-based models, the NST exhibits

a stark, progressive learning curve across the horizon. During the quieter September baseline (Period 1), it struggles with a 67.28% WMAPE, likely attempting to map complex attention vectors to relatively flat data. However, as the timeline enters the highly chaotic Black Friday and Christmas window (Periods 3 and 4), the Transformer's error drastically drops, ultimately achieving the lowest error of any model in the final period (**54.68%**). This empirically demonstrates that the De-Stationary Attention mechanism is uniquely specialized for capturing extreme, anomalous peak events, dynamically adjusting its mathematical scaling factors (τ , Δ) when standard models begin to falter.

The Classical Statistical Collapse A critical finding of our comparative analysis is the uniform breakdown of classical time-series models (SARIMAX and Prophet) across the entire horizon. Both models remain paralyzed at an unusable $\sim 90\%$ WMAPE regardless of the month. Because retail sparsity (intermittent zero-sales weeks) fundamentally breaks the continuity assumptions of Fourier series and ARIMA matrices, these models simply project a flat, mathematically compromised average. They completely fail to react to the Q4 volume spikes, confirming that traditional econometric techniques are structurally incompatible with highly skewed retail catalogs.

7 Improvements and Next Steps

7.1 Evolution from the Baseline Framework

The current forecasting engine represents a significant architectural evolution compared to the framework established by the previous research group. While the baseline provided a foundational understanding of the *Online Retail II* dataset, it suffered from structural limitations in terms of scalability, feature richness, and categorical handling. Our implementation introduces five pivotal improvements:

1. **Modular MLOps Architecture:** We transitioned from a single, monolithic research notebook to a fully modular, production-ready codebase. By adhering strictly to the DRY (Don't Repeat Yourself) principle, the data engineering, model training, and evaluation logic were decoupled into distinct, reusable Python modules, drastically improving both maintainability and system scalability.
2. **Integration of Rich Exogenous Signals:** We moved beyond univariate history by injecting dynamic external features, including UK national holidays, cyclical temporal encodings, and high-resolution pricing elasticity metrics.
3. **Transition to Cluster-Aware Modeling:** Instead of the computationally expensive and noisy per-SKU approach, we implemented a cluster-based strategy. This allows models to learn from aggregated behavioral patterns, significantly increasing the signal-to-noise ratio.
4. **Advanced Architectural Diversity:** We expanded the model library to include state-of-the-art algorithms such as the Non-Stationary Transformer (NST) and LightGBM with Tweedie loss, providing a more robust benchmark than the traditional SARIMA/Naive baseline.
5. **LLM-Driven Semantic Orchestration:** We replaced fragile, rule-based Regex parsing for product descriptions with a production-grade LLM orchestration (via Google Gemini API). This allows for 768-dimensional latent vector embeddings, enabling sophisticated categorical grouping that was previously impossible.

7.2 Key Technical Enhancements

In this sub-section, we detail the specific quantitative and qualitative advancements achieved in this iteration.

1. **Massive Reduction in Computational Overhead:** The transition from a per-SKU modeling logic to a cluster-based architecture has redefined the pipeline's efficiency. The baseline framework required approximately **60 minutes** to execute the training of its 1,000 individual models. Our optimized pipeline, despite utilizing more complex architectures like Transformers, completes the entire execution, from preprocessing to multi-model inference, in under **5 minutes** cumulatively. This represents a **91.67% reduction** in computational load, enabling near-real-time retraining cycles in a production MLOps environment.
2. **Comparative Performance and Structural Robustness:** We benchmarked our results against the previous group's performance (summarized in Table 9). A critical observation in

time-series forecasting is that per-SKU models often appear to have a theoretical advantage in "fitting" specific item-level noise. However, our cluster-based approach demonstrates superior generalization. Despite the inherent challenge of predicting aggregated shapes, our models outperform the baseline in the majority of cases. Notably, our **LightGBM (57.13% WMAPE)** and **NS-Transformer (60.81% WMAPE)** significantly surpass the previous group's best results (Global WMAPE of 71.36% and SARIMA at 61.79%). This proves that the inclusion of exogenous features and semantic context more than compensates for the loss of individual SKU-level granularity.

Model Architecture	Current Project (Ours)		Previous Group	
	WMAPE	Med. MAPE	WMAPE	Med. MAPE
LightGBM	57.13%	52.46%	74.24%	65.66%
NS-Transformer (SOTA)	60.81%	55.86%	N/A	N/A
Ridge Regression	60.99%	56.54%	N/A	N/A
SARIMAX	90.28%	79.61%	61.79%	57.24%
Prophet	89.37%	78.46%	N/A	N/A
Global Benchmark (Avg)	60.99%	56.54%	71.36%	65.52%

Table 9: Head-to-Head Performance Comparison: Current Project vs. Previous Baseline. Bold values indicate the winning architecture for each metric. *N/A* denotes models not implemented by the previous group.

7.3 Next Steps and Strategic Roadmap

To further evolve the predictive accuracy and operational utility of the system, we propose the following research directions:

- **Hierarchical Forecast Reconciliation:** Implementing a Minty or Top-Down reconciliation strategy to align our cluster-level forecasts with individual SKU-level constraints, potentially capturing the "best of both worlds."
- **Inventory-Aware Objective Functions:** Modifying the loss functions (e.g., asymmetric pinball loss) to prioritize the minimization of "Under-Forecasting" (which leads to stock-outs) over "Over-Forecasting" (which leads to warehouse costs).
- **Real-Time Demand Sensing:** Integrating the Agentic AI layer with real-time API feeds (e.g., current web traffic or social media trends) to adjust 12-week forecasts based on intra-week behavioral shifts.
- **Automated Ensembling (Stacking):** Developing a meta-model that dynamically weights the predictions of LightGBM, SARIMAX, and NST based on the specific uncertainty interval of each SKU cluster.

8 Team

We extend our sincere gratitude to everyone who guided and supported us throughout this project. This work is a proud reflection of our shared dedication, passion, and teamwork.



Federico Giorgi

fg2617



Luca Barattini

lb3656



Zoe Lucas

zcl2114

A Appendix: Individual SKU Error Spread Distributions

The following boxplots illustrate the distribution of Individual SKU Percentage Error (APE) and Absolute Volume Error (Physical Units) across the 1,000 active products for each of the four periods comprising the 12-week out-of-sample test set (Q4 2011). These visualizations provide granular insight into the variance, outliers, and temporal stability of each forecasting architecture when faced with the extreme Pareto-skew of the retail catalog.

A.1 LightGBM (State-of-the-Art Tree Baseline)

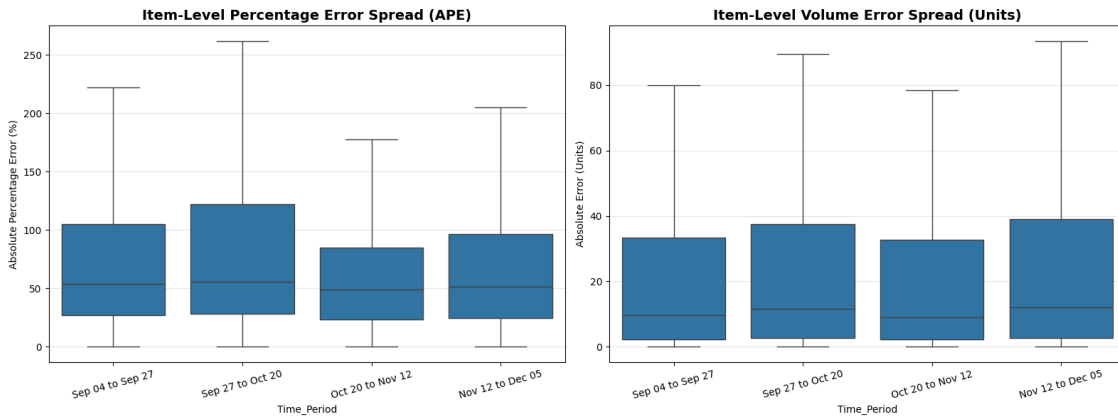


Figure 14: LightGBM: The Tweedie objective function heavily compresses the Absolute Percentage Error (APE) spread, successfully managing the intermittent nature of the long-tail catalog while maintaining tight volume accuracy across the entire Q4 transition.

A.2 Non-Stationary Transformer

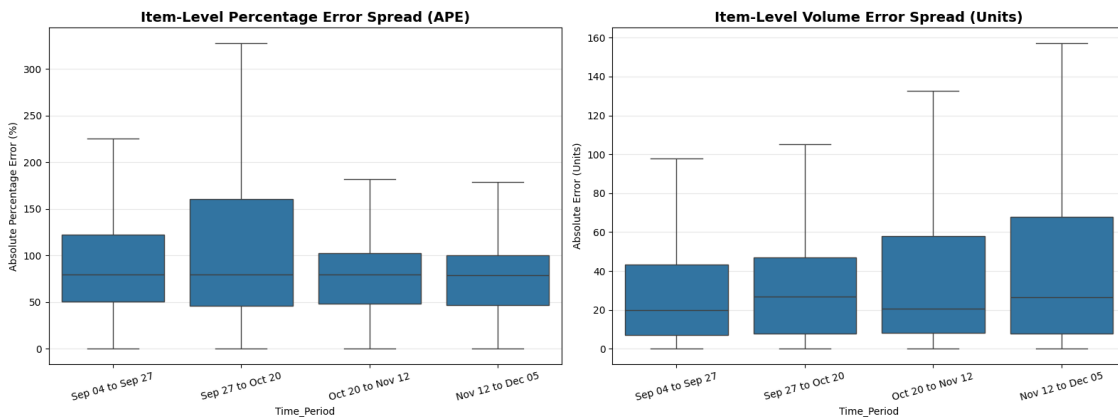


Figure 15: NS-Transformer: Visual confirmation of the model's progressive learning curve. The variance of both APE and Absolute Error visibly tightens as the timeline enters the highly volatile Black Friday and Christmas shopping weeks (Periods 3 and 4).

A.3 Ridge Regression

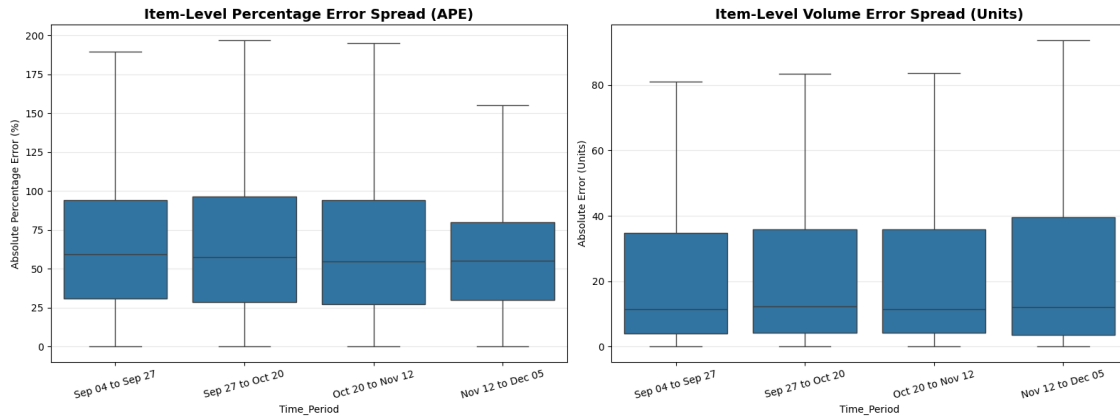


Figure 16: Ridge Regression (Baseline): The globally optimized α parameter and strict autoregressive lags keep the error variance constrained, establishing a remarkably stable linear baseline across the 12-week horizon.

A.4 SARIMAX

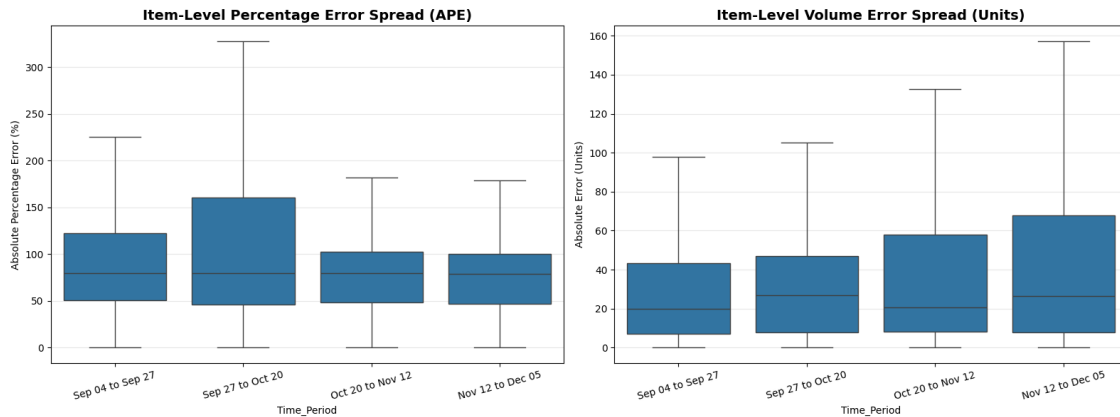


Figure 17: SARIMAX: The extreme sparsity of the retail dataset causes structural convergence issues. This results in massive variance in the percentage error distributions, as the rigid autoregressive matrices struggle to map intermittent, zero-inflated demand.

A.5 Facebook Prophet

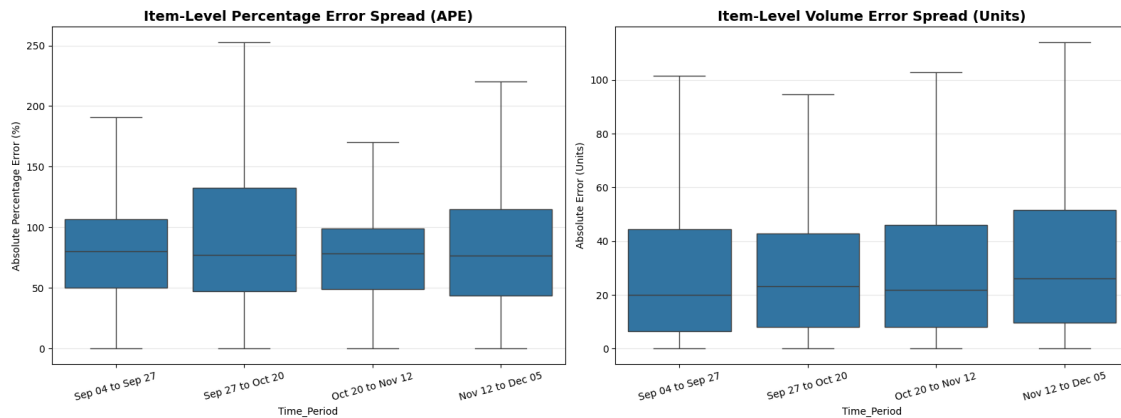


Figure 18: Facebook Prophet: Standard continuous Fourier curves struggle to capture isolated promotional spikes and sudden stock-outs, leading to a consistently wide spread of both percentage and absolute volume errors across the entire catalog.